Chapter 19
Information Visualization

Abstract In this chapter we describe the many GUI libraries on GNU/Linux systems. These include, GTK, Qt, as well as wxWidget and Fox Toolkit. We present OpenGL through many examples, which also present GLUT, GLUI and show example of using OpenGL from within Python. Graphics rendering engines (OGRE) and OpenGL helper libraries are also discussed. In addition to 3d graphics, graphics layout are also available using the Graphviz `dot` tool. Plotting software Gnuplot, and vector drawing tools Xfig and Inkscape are also discussed. Raytracing with PovRay is shown with the help of examples in Section 19.9. Programmatic creation of graphics is shown with the help of `gd` library, and the Asymptote library. Graphics visualization with GeomView, HippoDraw is described in Section 19.15. Multi-dimensional data visualization with GGobi is discussed. High-performance scientific data visualization with ParaView and OpenDX are discussed.

Contents

19.1 Graphical User Interfaces **412**
19.2 OpenGL **427**
19.3 OGRE : OO Graphics Rendering Engine **434**
19.4 Graphviz: dot **435**
19.5 gnuplot **436**
19.6 Grace/Xmgr **437**
19.7 Xfig **438**
19.8 Inkscape **439**
19.9 PovRay : Ray Tracing **440**
19.10 gd (graphics drawing) **442**
19.11 asymptote **443**
19.12 FreeType : Font Rendering **445**
19.13 Anti-grain geometry : AGG **446**
19.14 Geomview **446**
19.15 HippoDraw **448**
19.16 GGobi : multi-dimensional visualization **448**
19.17 ParaView and VTK **449**
19.18 OpenDX **452**
19.19 Conclusion **454**

S. Koranne, *Handbook of Open Source Tools*,
DOI 10.1007/978-1-4419-7719-9_19, © Springer Science+Business Media, LLC 2011

In Chapter 19 we discuss the various software tools and libraries available for information visualization. We discuss both creation of images and pictures for presentation as well rendering infrastructure and graphical user interfaces (GUI). Specifically we discuss, Qt, OpenGL, 'dot', gnuplot, `xfig`, `gd`, `asymptote`. Complete applications for graphics creation include Inkscape, ParaView, Geomview, HippoDraw and OpenDX for information presentation.

19.1 Graphical User Interfaces

We discuss graphical user interface systems that are available. In Section 19.1.2 we present the Gimp Tool Kit (gtk), Nokia/Trolltech's Qt is discussed in Chapter 19.1.3. WxWidgets is briefly discussed in Chapter 19.1.5.1 and the Fox toolkit is presented in Chapter 19.1.5. OpenGL front-end GUIs are presented for completeness towards the end of this part in Chapter 19.2.

19.1.1 X Window System

The X Window System is a graphical computer system and network protocol which is pervasive on GNU/Linux systems. X Window System provides the lowest level of hardware abstraction layer, and provides infrastructure for graphical user interfaces across networked computers. It allows for device independence, as well as remote display of screen contents over networks. Since it was designed for thin-clients the client side processing for X is very efficient, and places little demands on the computer (both CPU as well as memory). As a hardware abstraction layer, it provides windowing (including primitives to draw points, lines, polygons, and rasterization) as well as abstraction for user input (including keyboard and mouse). A singular feature of X is its operating system independence across the network. Since X only provides the lowest level primitives, other software such as window managers, or desktop environments such as GNOME and KDE usually is used on top of X11. GUI toolkits, when running on the X Window System make heavy use of the Xlib API. We present a number of such GUI libraries below, and although low level programming in Xlib is not only possible, but also considered to be more efficient, the convenience offered by using a GUI more than offsets the small gain in performance.

It should also be noted, that there also exist a number of problems with X11, which can be solved using external applications. For example, currently it is not possible to detach a client from one server to another, although by using VNC (see Section 1.7.1) stateful sessions on X11 are possible. Moreover, X protocol traffic is sent unencrypted over the network. This can be solved by using OpenSSH tunneling (see Section 1.5), which encrypts all traffic over a designated port.

19.1.2 GIMP Toolkit: GTK

In Section 14.5 we have presented the GIMP (GNU Image Manipulation Program). GIMP Toolkit (GTK) is a cross-platform widget toolkit for designing and implementing graphical user interfaces (GUI) for the UNIX X-Window system. It was initially created for GIMP, but now has expanded significantly in its usage, most importantly, it is the widget toolkit used for GNOME.

GTK is an object oriented (OO) widget toolkit written in the C programming language. The OO is implemented using the GLib object system (GObject). The underlying drawing primitives used by GTK depend on the platform, on UNIX it uses the X-windows xlib system to draw contents on the screen. Like the other toolkits we discuss in this part of the book, GTK admits many language bindings, including but not limited to C, C++, Python, and Java. Consider the example program shown in Listing 19.1.

19.1.2.1 Hello, World in GTK

Consider the following program in C language.

```
/* GTK Hello, World! program.(C) Sandeep Koranne, 2010 */
#include <gtk/gtk.h>
int main (int argc, char *argv[]) {
  GtkWidget *window;
  GtkWidget *label;
  char *markup, *str = "Hello, World!";
  gtk_init (&argc, &argv);
  window = gtk_window_new (GTK_WINDOW_TOPLEVEL);
  gtk_window_set_title (GTK_WINDOW (window), str);
  g_signal_connect (window, "destroy",
        G_CALLBACK (gtk_main_quit), NULL);
  markup = g_markup_printf_escaped("<span size=\"x-large\""
  "style=\"italic\">%s</span>", str);
  label = gtk_label_new (str);
  gtk_label_set_angle( GTK_LABEL( label ), 45.0 );
  gtk_label_set_markup( GTK_LABEL( label ), markup );
  gtk_container_add (GTK_CONTAINER (window), label);
  gtk_widget_show_all (window);
  gtk_main ();  return 0;
}
```

Listing 19.1 Using GTK GUI Libraries

We compile this program using the `pkg-config` tool as:

```
gcc gtk_hw.c -o gtk_hw `pkg-config --cflags gtk+-2.0` \
    `pkg-config --libs gtk+-2.0`
```

Running this program gives us the picture as shown in Figure 19.1.

The example above shows the use of Pango Text Attribute Markup Language. Using this mechanism the attributes of the text elements can be changed.



Fig. 19.1 Hello, World with GTK showing Pango

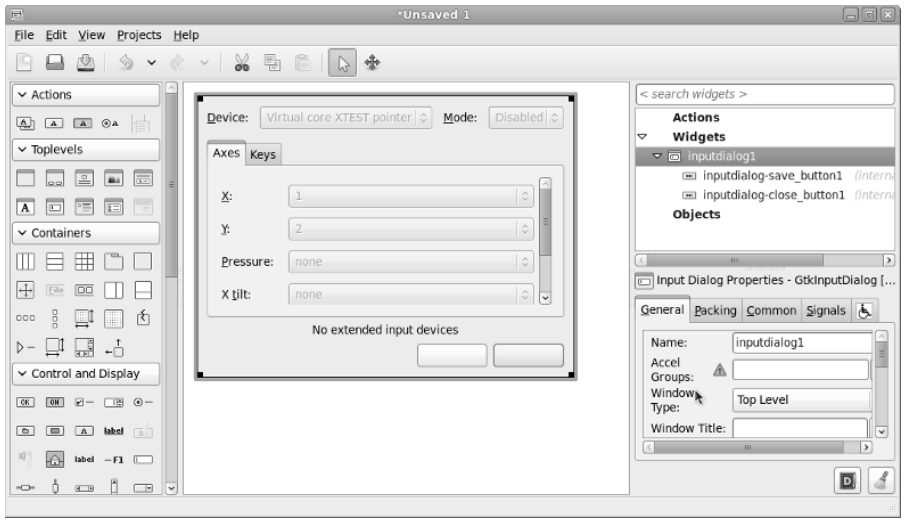


Fig. 19.2 Glade: user interface designer for GTK

19.1.2.2 Using Glade: User Interface Designer

Designing user interfaces by compositing widgets by hand gets tedious very quickly, thus most widget systems have developed GUI tools to design GUIs. For GTK, the user interface designer is called Glade. An example is shown in Figure 19.2. Using Glade a GUI application using GTK can be built very quickly.

19.1.3 Qt: Application development framework

Nokia/Trolltech's Qt application framework library started out as a graphical user interface widget library, but over the years has grown to include many features which are not traditionally associated with GUIs. These include threading, internationalization, XML processing and much more. In this chapter we discuss the Qt framework. In the example we have used Qt version 4.6.2. A picture of Qt Creator is shown in Figure 19.3.

Qt is written in C++, but program bindings exist for many other programming languages such as Python. Qt is also cross-platform, meaning the same source code

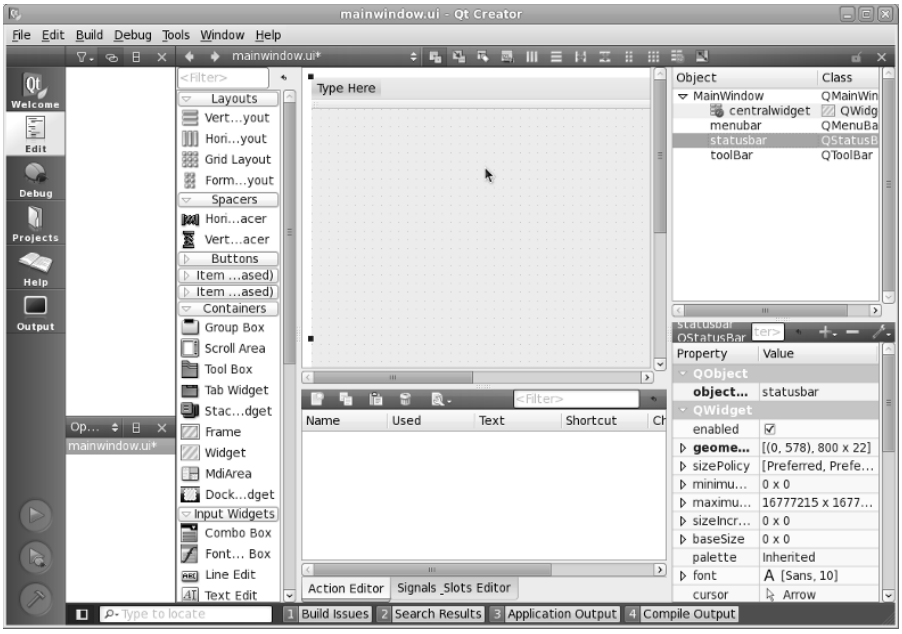


Fig. 19.3 Qt Creator

can be compiled on UNIX, GNU/Linux, Microsoft Windows, Apple Mac OS X, even embedded Linux. In addition to GUI widgets Qt also includes functionality required by application developers to create rich and functional products. Qt is written in an Object Oriented manner (as most GUI toolkits are), and has a unique *signal* and *slot* mechanism which connects user actions to program behavior, see 19.1.3.2.

Starting with Qt 4, the organization of Qt has been changed to consist of several modules, each of which can be used independently (except the Core module). The following modules are most useful to application developers:

19.1.3.1 Qt 4 Module Architecture

- QtCore : Core non-GUI classes,
- QtGUI : GUI interface and widgets,
- QtNetwork : network programming,
- QtOpenGL : OpenGL programming,
- QtSql : database integration using SQL,
- QtSVG : displaying contents of scalable vector graphics (SVG) files,
- QtXML : XML parser and support.

There are other modules which deal with web presentation, multimedia framework, UI testing, multi-threading, Qt 3 support, and dynamic UI creation.

Consider the program presented below:

```
// Hello World from Qt
// (C) Sandeep Koranne, 2010
```

```
#include <QApplication>
#include <QLabel>

int main( int argc, char *argv[] )
{
  QApplication app( argc, argv );
  QLabel *label = new QLabel( "Hello, World from Qt!", 0 );
  label->show();
  return app.exec();
}
```

Listing 19.2 Example of using Qt GUI Libraries

To compile this program using Qt 4, we do:

```
$qmake -project
$qmake
$make
```

The constructed project file from Qt is instructive:

```
TEMPLATE = app
TARGET =
SOURCES += qhw.cpp
```

Fig. 19.4 Hello World in Qt



19.1.3.2 Qt Signals and Slots

Before Qt, existing GUI libraries used the concept of *callback* functions (c.f. Motif) or event listeners (main loop) to connect user interaction with program computation. Qt uses an unique C++ pre-processor based solution called signal and slots. Using the `QObject::connect` function, an user can *connect* a *signal* such as button click, menu shortcut to a *slot* such as button-pressed. The Qt framework manages the internal connections between the defined signal and slot connections.

```
// Qt signal and slot example
#include <QApplication>
#include <QLabel>
#include <QSpinBox>
#include <QHBoxLayout>

int main( int argc, char *argv [] ) {
  QApplication app( argc, argv );
  QWidget window;
  QHBoxLayout *hbox = new QHBoxLayout( &window );
  QLabel   *label = new QLabel("0");
  QSpinBox *spbox = new QSpinBox;
  hbox->addWidget( spbox );
```

```
  hbox->addWidget( label );
15  QObject::connect( spbox, SIGNAL( valueChanged(int) ),
        label, SLOT( setNum(int) ) );
  window.show();
  return app.exec();
}
```

Listing 19.3 Signal/slot mechanism in Qt

The output of the program is shown in Figure 19.5.



Fig. 19.5 Signals and slots example with spinbox

The signal and slots definition in derived classes is not C++ per se, but has to be pre-processed by the `moc` *meta object compiler* from Qt. This compiler generates appropriate code for the class to enable signal and slot connections.

Qt comes complete with instructive examples, and using the Qt Creator and the examples, many complex products have been developed, as an example consider the Qt frontend to Octave as shown in Figure 19.6.

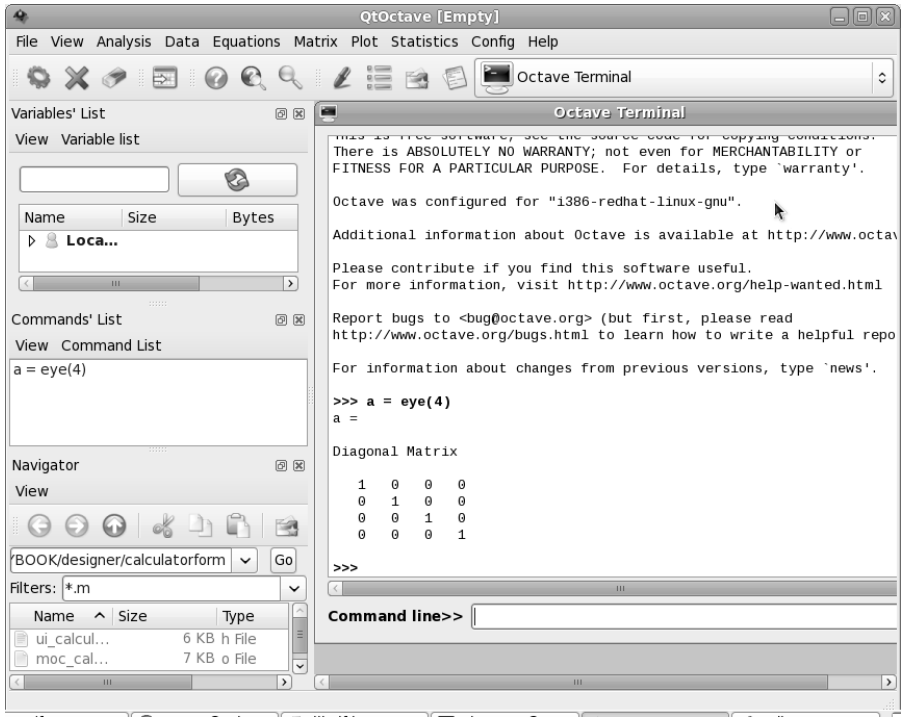


Fig. 19.6 QtOctave frontend to Octave

19.1.4 Qt's application programming API

In addition to providing graphical user interface widgets, Qt API also provides general purpose application development APIs such as database access, inter-process communication, shared memory, multi-threading, Webkits (for rendering multimedia), and much more. We present an example of some of the application development APIs below.

19.1.4.1 Shared Memory

Consider the procedure for shared memory transactions between related processes (which know the *key* to the shared memory):

```
#include <qsharedmemory.h>
{
  QSharedMemory sharedMemory;
  sharedMemory.create(size);
  sharedMemory.lock();
  char *to = (char*)sharedMemory.data();
  const char *from = buffer.data().data();
  memcpy(to, from, qMin(sharedMemory.size(), size));
  sharedMemory.unlock();
  sharedMemory.detach();
}
```

Another API for shared memory processing is given in Section 5.8, which uses the Apache Portable Runtime (apr) functions.

19.1.4.2 Network programming using Qt API

To use the TCP networking APIs in Qt, the TCP header files:

```
#include <QTcpServer>
#include <QTcpSocket>
```

should be included in the application, and the `network` library should be added to the `QT` section of the 'qmake' file. To create a TCL server, the `QTcpServer` object and an associated `QTcpSocket` should be created. To start *listening* to a connection the server socket's `isListening()` member function should be invoked. Thereafter, the client can be connected to the server using the `connectToHost` member function of the client socket.

19.1.5 Other GUI Toolkits

In addition to GTK and Qt, several other GUI toolkits find niche use in particular domains. These may exist for a number of reasons, including licensing restriction and developer familiarity.

19.1.5.1 WxWidgets

An introductory program in WxWidgets is shown below.

```
#include "wx/wx.h"
class MyApp: public wxApp {virtual bool OnInit();};
class MyFrame: public wxFrame {
public:
    MyFrame(const wxString& title, const wxPoint& pos, const wxSize& size);
    void OnQuit(wxCommandEvent& event);
    void OnAbout(wxCommandEvent& event);
    DECLARE_EVENT_TABLE()
};

enum { ID_Quit = 1, ID_About, };

BEGIN_EVENT_TABLE(MyFrame, wxFrame)
    EVT_MENU(ID_Quit, MyFrame::OnQuit)
    EVT_MENU(ID_About, MyFrame::OnAbout)
END_EVENT_TABLE()

IMPLEMENT_APP(MyApp)

bool MyApp::OnInit() {
    MyFrame *frame = new MyFrame( _("Hello World"), wxPoint(50, 50),
                                  wxSize(450,340) );
    frame->Show(true);
    SetTopWindow(frame);
    return true;
}

MyFrame::MyFrame(const wxString&title,const wxPoint&pos,const wxSize&size)
: wxFrame( NULL, -1, title, pos, size ) {
    wxMenu *menuFile = new wxMenu;
    menuFile->Append( ID_About, _("&About...") );
    menuFile->AppendSeparator();
    menuFile->Append( ID_Quit, _("E&xit") );
    wxMenuBar *menuBar = new wxMenuBar;
    menuBar->Append( menuFile, _("&File") );
    SetMenuBar( menuBar );
    CreateStatusBar();
    SetStatusText( _("Welcome to wxWidgets!") );
}

void MyFrame::OnQuit(wxCommandEvent& WXUNUSED(event)){Close(TRUE);}

void MyFrame::OnAbout(wxCommandEvent& WXUNUSED(event)) {
    wxMessageBox( _("This is a wxWidgets Hello world sample"),
                  _("About Hello World"),
                  wxOK | wxICON_INFORMATION, this);
}
```

Listing 19.4 Using wxWidgets GUI library

The resulting GUI is shown in Figure 19.7. A more complex example of GUI development with WxWidgets is the `wxmaxima` program as shown in Figure 17.2.

19.1.5.2 Java SWT

In addition to C/C++ based GUI construction, Java also has extensive graphical facilities as embodied in the Java SWT toolkit.



Fig. 19.7 Hello World with WxWidgets

19.1.5.3 FOX Toolkit

FOX stands for Free Objects for X. It is a C++ based class library for GUI. Example of a FOX developed UI application can be seen in Figure 19.8.

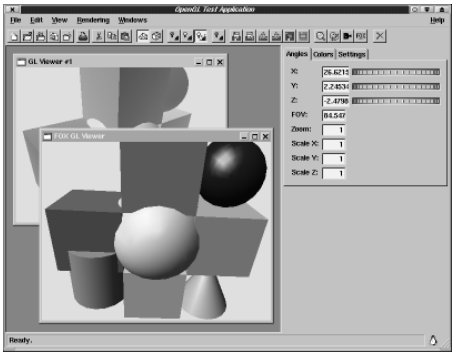


Fig. 19.8 glview example for FOX

19.1.5.4 GUI Development using Tcl/Tk

An example of using Tcl/Tk for GUI development is shown in Figure 3.1, which implements a complete CVS/SVN front-end using Tk. A simpler example of using Tcl/Tk is shown in Section 1.6.6 which shows a push-button implemented using Tcl/Tk.

19.1.5.5 GUI Development using PyQt

Using Qt from Python is straightforward using the Python bindings. Consider the example shown in Listing 19.5.

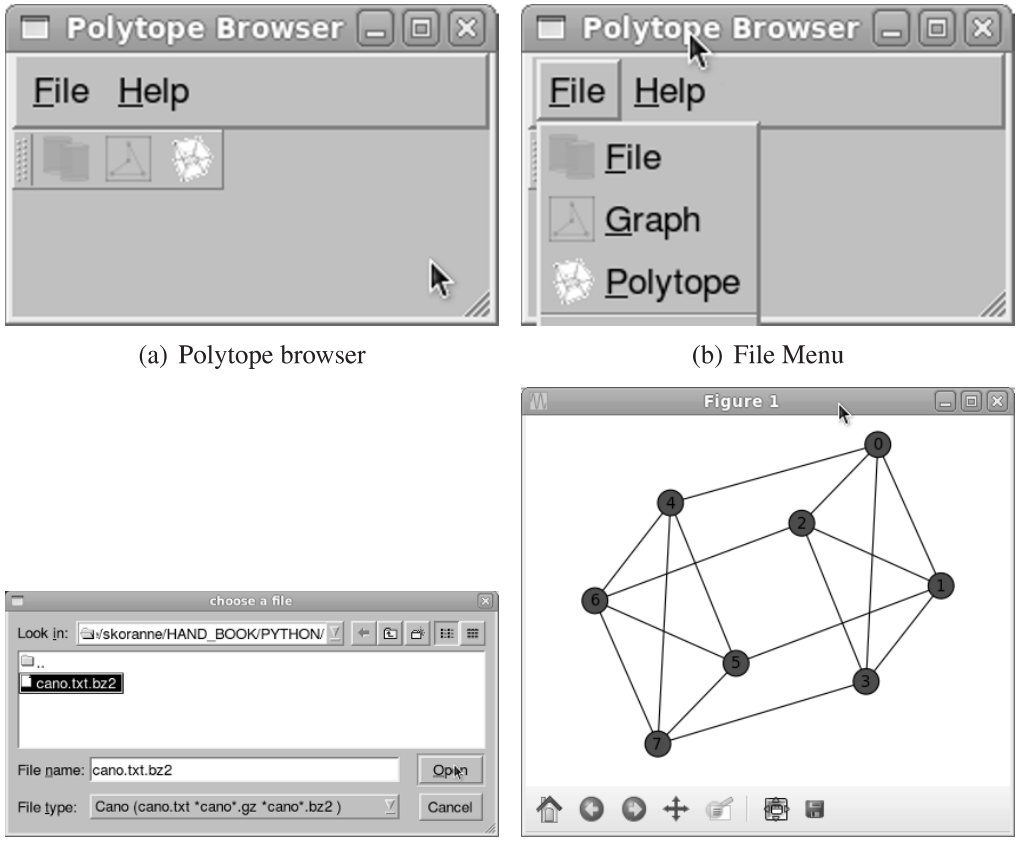
```
#!/usr/bin/python
# \file pyqt.py
# \author Sandeep Koranne
```

```
import sys
from PyQt4 import QtGui
app = QtGui.QApplication(sys.argv)

widget = QtGui.QWidget()
widget.resize(400, 200)
widget.setWindowTitle('Main Window in PyQT')
widget.show()
sys.exit(app.exec_())
```

Listing 19.5 Example of PyQt

A larger example of PyQt using several Qt functions as well as the Networkx graph toolkit is shown in Listing 19.6.



(a) Polytope browser (b) File Menu

(c) File open dialog (d) Diagram of polytope

Fig. 19.9 A polytope browser written in PyQt

```
#!/usr/bin/python
# \file main.pyw
# \author Sandeep Koranne

try:
    import matplotlib.pyplot as plt
    import sys
    import subprocess
```

```
except:
    raise

import networkx as nx

import sys
import gzip
import bz2

import sys
from qt import *

dimension = sys.argv[1]


class Polytope:
    """Facets, Fvector, signature and parent information"""
    def __init__(self):
        self.facets = []
        self.fvec = []
        self.signature = ""
        self.parent = 0
        self.parendId = -1
        self.cut_set = []
    def dump(self):
        print self.signature
        print " "
        print self.facets
        print " "
        print self.parent
        print " "
        print self.parentId
    def recreate_format( self, fileName ):
        fs = open( fileName, 'wt' )
        fs.write('VERTICES_IN_FACETS\n')
        fs.write('\n')
        for i in self.facets:
            fs.write('{ ')
            for j in i:
                fs.write( str(j) )
                fs.write(' ')
            fs.write(' }\n')
        fs.write('\n')
        fs.write('F_VECTOR ( ')
        for i in self.fvec:
            fs.write( str(i) )
            fs.write(' ')
        fs.write(' )\n\n ')
        fs.write( self.signature )
        fs.write( ' Parent HC = [ ')
        fs.write( self.parent )
        fs.write( ' ] ')
        fs.write( ' [ ')
        for i in self.cut_set:
            fs.write( str(i) )
            fs.write(' ')
        fs.write( ' ]\n')
        fs.close()

def IndexPolytopes( polytopes ):
    """Given a list of polytopes make a dictionary with the index"""
    poly_dict = {}
    i = 0;
    for p in polytopes:
        poly_dict[ p.signature ] = i
        i=i+1
```

```
    for p in poly_dict:
        me = poly_dict[p] # index of myself
        my_parent_signature = polytopes[me].parent
        my_parent_id = poly_dict[ my_parent_signature]
        polytopes[me].parentId = my_parent_id

def ParseFile( fileName ):
    print "Reading file %s" %fileName
    if fileName.find( '.gz' ) != -1:
        file = gzip.open( fileName, 'rb' )
        lines = file.readlines()
        file.close()
    elif fileName.find( '.bz2' ) != -1:
        file = bz2.BZ2File( fileName, 'rb' )
        lines = file.readlines()
        file.close()
    else:
        f = open( fileName, 'r' )
        lines = f.readlines()
        f.close()

    expectFacets = False
    expectFVec = False
    expectSignature = False
    i = 0
    allPolytopes = []
    for l in lines:
        if l == 'VERTICES_IN_FACETS\n':
            if i > 0:
                allPolytopes.append( next )
            next = Polytope()
            i=i+1
            expectFacets = True
        elif l == '\n':
            if expectFacets:
                expectFacets = False
                expectFVec = True
            elif expectFVec:
                expectFVec = False
                expectSignature = True
            elif expectSignature:
                expectSignature = False
                expectFacets = True

        else: #Data to be processed
            if expectFacets:
                # data is in { 1 2 3 4 }
                face_str = l.strip()[1:-1]
                face = []
                for vid in face_str.split(' '):
                    try:
                        face.append( int(vid) )
                    except:
                        pass
                next.facets.append( face )
            elif expectFVec:
                head_string = l[0:8]
                assert head_string == 'F_VECTOR'
                for num in l[8:].split(' '):
                    try:
                        next.fvec.append( int(num) )
                    except:
                        pass
            elif expectSignature:
                # break up the line into components
                parent_hc = l.find('Parent HC =')
```

```
                assert parent_hc > 0
                next.signature = l[0:parent_hc].strip()
                parent_hc_end = l[parent_hc+14:].find(']')
                offset = parent_hc+14+parent_hc_end
                try:
                    next.parent = l[parent_hc+14:offset].strip()
                except:
                    print "Error in parsing parent id"

                # we have read the parent, now read the cut set
                for cut_set in l[offset+1:].split(' '):
                    try:
                        next.cut_set.append( int( cut_set ) )
                    except:
                        pass

    allPolytopes.append( next )
    print "Collected %d polytopes"%len( allPolytopes )
    IndexPolytopes( allPolytopes )
    return allPolytopes


def CalculateHeritage( P, i ):
    """Given the polytope table and the  polytope index find its heritage"""
    answer = []
    P[i].dump()
    parentId = P[i].parentId
    while i != parentId:
        answer.append( (parentId, P[i].cut_set ) )
        i = parentId
        parentId = P[i].parentId


    return answer

class MainWindow(QMainWindow):

    def __init__(self, *args):
        apply(QMainWindow.__init__, (self, ) + args)
        self.setCaption("Polytope Browser")

  #load icons from PNG files 60x60 size
  self.fileOpenIcon = QPixmap("db_icon.png")
        self.graphIcon    = QPixmap("graph_icon.png")
        self.polytopeIcon = QPixmap("poly_icon.png")


        self.actionLoadPolytope = QAction( self, "Polytope" )
        self.actionLoadPolytope.setText("Load polytope db")
        self.actionLoadPolytope.setMenuText("&File")
        self.actionLoadPolytope.setStatusTip("Load a polytope db file")
        self.actionLoadPolytope.setIconSet( QIconSet( self.fileOpenIcon ) )
        self.connect( self.actionLoadPolytope,SIGNAL("activated()"),
                      self.loadPolytopeDB )

        self.actionGraph = QAction( self, "Graph" )
        self.actionGraph.setText("Calculate graph")
        self.actionGraph.setMenuText("&Graph")
        self.actionGraph.setStatusTip("Compute 1-skeleta (graph)")
        self.actionGraph.setIconSet( QIconSet( self.graphIcon ) )
        self.connect( self.actionGraph,SIGNAL("activated()"),
                      self.computeGraph )


        self.actionPoly = QAction( self, "Poly" )
        self.actionPoly.setText("Analyze polytope")
        self.actionPoly.setMenuText("&Polytope")
```

```
        self.actionPoly.setStatusTip("Analyze polytope")
        self.actionPoly.setIconSet( QIconSet( self.polytopeIcon ) )

        self.statusBar = QStatusBar( self )

        self.fileMenu = QPopupMenu()
        self.actionLoadPolytope.addTo( self.fileMenu )
        self.actionGraph.addTo( self.fileMenu )
        self.actionPoly.addTo( self.fileMenu )

        self.quitAction = QAction( self, "Quit" )
        self.quitAction.setText("Quit")
        self.quitAction.setMenuText( "&Quit" )
        self.fileMenu.insertSeparator()
        self.quitAction.addTo( self.fileMenu )
        self.connect( self.quitAction, SIGNAL("activated()"), self.quitApp )

        self.helpMenu = QPopupMenu()
        self.actionAboutQt = QAction( self, "AboutQt" )
        self.actionAboutQt.addTo( self.helpMenu )
        self.connect( self.actionAboutQt,
                      SIGNAL("activated()"),
                      self.slotAboutQt )

        self.menuBar().insertItem( "&File", self.fileMenu )
        self.menuBar().insertItem( "&Help", self.helpMenu )

        self.toolBar = QToolBar( self, "Compute" )
        self.actionLoadPolytope.addTo( self.toolBar )
        self.actionGraph.addTo( self.toolBar )
        self.actionPoly.addTo( self.toolBar )

    def slotAboutQt( self ):
        QMessageBox.aboutQt( self )

    def loadPolytopeDB( self ):
        self.polytope_db_filename = QFileDialog().getOpenFileName(
            ".",
            "Cano (cano.txt *cano*.gz *cano*.bz2 )",
            self,
            "open file dialog",
            "choose a file" )
        if self.polytope_db_filename:
            print "Loading polytope file"
            print self.polytope_db_filename
            self.parsed_db = ParseFile( self.polytope_db_filename )
            self.number_of_polytopes = len( self.parsed_db )

    def gotPolytopeId( self ):
        self.current_polytope_id = int( self.spinBox.value() )
        self.number_input.hide()
        self.constructGraph()

    def constructGraph( self ):
        print "Calcuating graph of polytope id %d"
        %(self.current_polytope_id)
        heritage = CalculateHeritage( self.parsed_db,
                                      self.current_polytope_id )
        for h in heritage:
            print h
        P = self.parsed_db[ self.current_polytope_id ]
        out_file_name = 'g.cano'
        P.recreate_format( out_file_name )
        p0 = subprocess.Popen(['~/cano_proc','-f',out_file_name,
                               '-d',dimension,'-G','-m 1'],shell=False,
                              stdin=subprocess.PIPE,
                              stdout=subprocess.PIPE)
```

```
        rc = p0.wait()
        if rc != 0:
            print "Graph construction from %s failed." %out_file_name
        GORIG=nx.read_adjlist("graph.txt", nodetype=int)
        G=GORIG.copy()
        mylabels={}
        for v in G.nodes():
            mylabels[v]=int(v)
        diam = nx.diameter(G)
        print "Diameter of G = %d"%diam
        #    b=nx.betweenness_centrality(G)
        #    for v in G.nodes():
        #        print "%0.2d %5.3f"%(v,b[v])
        #pl = plt.subplot(221)
        nx.draw(GORIG, labels=mylabels,pos=nx.spring_layout(GORIG))
        plt.savefig("san.png") # save as png
        plt.show() # display

    def computeGraph( self ):
        self.number_input = QDialog( self )
        layout = QHBoxLayout( self.number_input )
        self.number_input.setCaption( "Enter polytope number: ")
        self.spinBox = QSpinBox( self.number_input )
        layout.addWidget( self.spinBox )
        self.spinBox.setRange( 0, self.number_of_polytopes-1 )
        self.pnum_button = QPushButton( self.number_input, "Accept")
        layout.addWidget( self.pnum_button )
        self.pnum_button.setText("Accept")
        self.connect( self.pnum_button, SIGNAL("clicked()"),
                      self.gotPolytopeId )
        self.number_input.show()

    def quitApp( self ):
        # close all files and processes
        exit(0)
    number_of_polytopes = 0
    current_polytope_id = 0
    polytope_db_filename = ""
    parsed_db = None

def main(args):
    app=QApplication(args)
    win=MainWindow()
    win.show()
    app.connect(app, SIGNAL("lastWindowClosed()"), app,
                SLOT("quit()") )
    app.exec_loop()

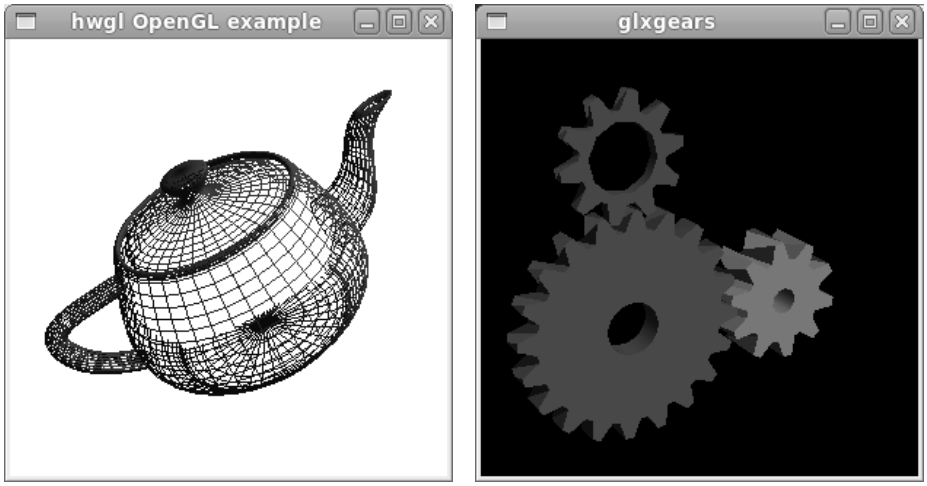
if __name__=="__main__":
        main(sys.argv)
```

Listing 19.6 Qt integration with Python, polytope browser

The program in Listing 19.6 when run, produces the GUI as shown in Figure 19.9(a). The File menu as well as the file dialog are shown in Figure 19.9(b) and (c), respectively. The Python program uses the `matplot` module which is used to draw the diagram as shown in Figure 19.9(d).

19.2 OpenGL

OpenGL is an industry standard for implementing high-performance graphics rendering programs. It is a high-level API which provides window system and operating system independent software interface to the graphics hardware. An OpenGL compliant program can execute on any platform to produce identical graphics output. Moreover, each graphics hardware provider can implement optimized OpenGL implementations which aid in high-performance.



(a) OpenGL UTah Teapot (b) OpenGL gears example

Fig. 19.10 Classic example of OpenGL

OpenGL provides a number of facilities which can be used by the application program. These include:

1. Graphics primitives: primitives include:
   a. points (GL_POINTS),
   b. lines (GL_LINES),
   c. line-loop (GL_LINE_LOOP),
   d. line-strip (GL_LINE_STRIP),
   e. convex polygon (GL_POLYGON),
   f. quadrilaterals (GL_QUADS),
   g. quad-strip (GL_QUAD_STRIP),
   h. triangles (GL_TRIANGLES),
   i. triangle strip (GL_TRIANGLE_STRIP),
   j. triangle fan (GL_TRIANGLE_FAN).
2. Indexed and RGBA color mode: indexed color mode is generally not used as operations such as *fogging* are complicated, but it is nevertheless used in applications where *object picking* is required,

3. OpenGL pipeline and contexts: OpenGL is a state-machine based drawing system. The OpenGL state-machine state can be inspected and changed by the application. Most of the state pertaining to drawing such as line-width, shading, color is changed by using the API functions,
4. Images and textures: to provide realistic three-dimensional portrayal OpenGL supports application of textures to objects. Images can be transmitted to and from the hardware memory buffer,
5. Extensions provide NURBS and curves support:

Consider the program in Listing 19.7.

```
// \file hwgl.cpp
// \author Sandeep Koranne, (C) 2010
// \description Example of OpenGL primitives
#include <GL/glx.h>
#include <GL/gl.h>
#include <GL/glu.h>
#include <GL/glut.h>

GLfloat light_diffuse[] = {1.0, 0.0, 0.0, 1.0};
GLfloat light_position[]= {1.0, 1.0, 1.0, 0.0};

static int main_window = 0;
static GLuint renderModelList = 1;

static float x,y,z;
static float localScale = 1.0;
static const float MOTION = 1.0;

void processSpecialKeys( int key, int x, int y ) {
  switch( key ) {
  case GLUT_KEY_LEFT: { x += MOTION; break; }
  case GLUT_KEY_RIGHT:{ x -= MOTION; break; }
  case GLUT_KEY_UP:   { y -= MOTION; break; }
  case GLUT_KEY_DOWN: { y += MOTION; break; }
  }
  glutPostRedisplay();
  glFlush();
}


void processNormalKeys( unsigned char key, int x, int y ) {
  if( key == 27 ) { exit( 0 ); }
  if( key == 'b') { localScale /= 1.05f; }
  if( key == 'f') { localScale *= 1.05f; }
  glutPostRedisplay();
  glFlush();
}

void renderScene(void) {
  glClear( GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT );
  glPushMatrix();
  glRotatef( 45.0, 4.0, 2.0, 4.0 );
  glScalef( localScale, localScale, localScale );
  glCallList( renderModelList );
  glPopMatrix();
  glFlush();
}

static void CreateModel(void) {
  GLUquadricObj *obj = gluNewQuadric();
  gluQuadricDrawStyle( obj, GLU_FILL );
  renderModelList = glGenLists( 1 ); // get new display list
  glNewList( renderModelList, GL_COMPILE );
```

```
  glColor3f( 0.4, 0.4,0.6 );
  glutWireTeapot( 4.0 );
  glEndList();
}

void InitGL(void) {
  x=y=z=0;
  CreateModel();
  glLightfv( GL_LIGHT0, GL_DIFFUSE, light_diffuse );
  glLightfv( GL_LIGHT0, GL_POSITION, light_position );
  glEnable( GL_LIGHTING );
  glEnable( GL_LIGHT0 );
  glClearColor( 1.0, 1.0, 1.0, 0.5f );
  glEnable( GL_DEPTH_TEST );
  glMatrixMode( GL_PROJECTION ); // setup viewing system
  gluPerspective( 40.0,         // field-of-view
                  1.0,          // aspect ratio
                  1.0, 10.0 ); // Z-near and far
  glMatrixMode( GL_MODELVIEW ); // scene coordinates
  gluLookAt( 0.0, 0.0, 9,       // eye is at (0,0,9)
             0.0, 0.0, 0.0,     // world center at origin
             0.0, 1.0, 0.0 );   // Y is up
}

int main( int argc, char *argv [] ) {
  glutInit( &argc, argv ); // initialize GLUT sub-system
  glutInitDisplayMode( GLUT_DEPTH | GLUT_RGBA ); // use RGBA color
  main_window = glutCreateWindow("hwgl OpenGL example");
  InitGL();
  glutKeyboardFunc( processNormalKeys );
  glutSpecialFunc( processSpecialKeys );

  glutDisplayFunc( renderScene );
  glutMainLoop();
  return (0);
}
```

Listing 19.7 Drawing graphics with OpenGL

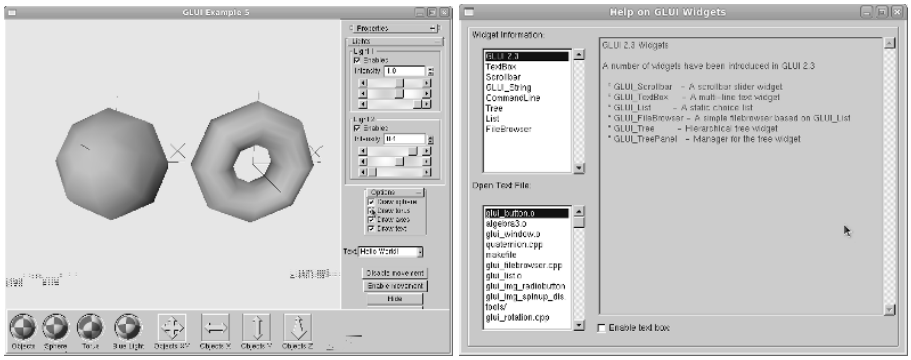
19.2.1 GLUT : OpenGL Utility Toolkit

GLUT is an utility toolkit for OpenGL which make the interaction between OpenGL and the underlying window system easier to manage as it provides the graphics context (GLX for X-Windows), as well as the initial drawing window where the context is attached. GLUT also supports a wide variety of input interfaces including keyboard, mouse and trackwheel. It also provides high-level graphics primitives such as the Utah teapot (as shown in Figure 19.10(a)) and the OpenGL `glxgears` program in Figure 19.10(b).

In addition to GLUT, also consider using GLFW (OpenGL contexts and input manager) which aims to provide operating-system independent access to OpenGL, similar to GLUT, but has been developed more recently.

19.2.2 GLUI : GUI for OpenGL

As stated above in Section 19.2, OpenGL by itself is window-system independent, and thus does not provide any graphical user interface as part of the standard. However, all major GUI toolkits have included support for OpenGL. There is also an independent completely OpenGL based user interface library, GLUI, which is described below.



(a) Applications, Internet and Office

Fig. 19.11 OpenGL GLUI examples

Consider the short program in the listing below. It shows a simple example of using OpenGL, GLUT and GLUI within the X-Window system. The libraries are available and ready to be used to create immersive graphics with the high performance coming from OpenGL.

```
////////////////////////////////////
// glexample.cpp
// (C) Sandeep Koranne, 2010
// Example of OpenGL with GLUT and GLUI
////////////////////////////////////
#include <GL/glx.h>
#include <GL/gl.h>
#include <GL/glu.h>
#include <GL/glut.h>
#include <GL/glui.h>

void InitGL(void);
void renderScene(void);
void processSpecialKeys( int key, int x, int y );
void processNormalKeys( unsigned char key, int x, int y );
void changeSize( int w, int h );
const int WIDTH = 600, HEIGHT = 600;
GLUI_StaticText *pStaticText = NULL;
int main_window = 0;
int xdim=0, ydim=0;
void myGlutMouse( int button, int button_state, int x, int y ) {
  //
}

void myGlutMotion( int x, int y ) {
   GLdouble mat[16];
```

```
    GLdouble proj[16];
    GLint viewport[4];
    glGetDoublev(GL_MODELVIEW_MATRIX,mat);
    glGetDoublev(GL_PROJECTION_MATRIX,proj);
    glGetIntegerv(GL_VIEWPORT,viewport);
     // our viewport is static
    GLdouble dx, dy, dz;
    gluUnProject(x,HEIGHT-y,0,mat,proj,viewport,&dx,&dy,&dz);
    char ncs[100];
    sprintf(ncs,"  X=%d,Y=%d",(int)dx,(int)dy);
    pStaticText->set_text((char*)ncs);
    glutPostRedisplay();
}

void myGlutIdle( void ) {
  if ( glutGetWindow() != main_window ) {
    glutSetWindow(main_window);
  }
  glutPostRedisplay();
}

int Initialize( int argc, char *argv[] ) {
  glutInit( &argc, argv );
  glutInitDisplayMode( GLUT_DEPTH | GLUT_SINGLE | GLUT_RGBA );
  glutInitWindowPosition( 100,100 );
  glutInitWindowSize( WIDTH, HEIGHT );
  main_window = glutCreateWindow("GlExample");
  InitGL();
  glutDisplayFunc( renderScene );
  glutMotionFunc( myGlutMotion );

  GLUI *gluiTop = GLUI_Master.create_glui_subwindow(
   main_window, GLUI_SUBWINDOW_TOP );
  GLUI *glui2 = GLUI_Master.create_glui_subwindow(
   main_window, GLUI_SUBWINDOW_LEFT );
  glui2->add_checkbox( "Display WireFrame");
  glui2->add_checkbox( "Black/White");

  glui2->set_main_gfx_window( main_window );
  pStaticText = glui2->add_statictext("X=0,Y=0");

  glutKeyboardFunc( processNormalKeys );
  glutSpecialFunc( processSpecialKeys );
  glutReshapeFunc( changeSize );
  glutMainLoop();
  return 0;
}

int main( int argc, char *argv[] ) {
  return Initialize( argc, argv );
}

double localScale = 1.0f;
void renderScene(void) {
  glClear( GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT );
  glLoadIdentity();
  glTranslatef( xdim, ydim, 0 );
  glScalef( localScale, localScale, localScale );
  glColor3f(0,0,0);
  glRecti( 0, 0 ,5,1);
  glColor3f( 1.0, 0.0, 0.0 );
  glRecti( 0, 1, 1,2 );
  glFlush();
}

void processSpecialKeys( int key, int x, int y ) {
  switch( key ) {
```

```
    case GLUT_KEY_LEFT: { xdim += 1; break; }
    case GLUT_KEY_RIGHT:{ xdim -= 1; break; }
    case GLUT_KEY_UP:   { ydim -= 1; break; }
    case GLUT_KEY_DOWN: { ydim += 1; break; }
    }
    glutPostRedisplay();
    glFlush();
}

void processNormalKeys( unsigned char key, int x, int y ) {
    if( key == 27 ) { exit( 0 ); }
    if( key == 'b') { localScale /= 2.0f; }
    if( key == 'f') { localScale *= 2.0f; }
    glutPostRedisplay();
    glFlush();
}

void InitGL(void) {
    glClearColor( 1.0, 1.0, 1.0, 0.5f );
    glClearDepth( 1.0 );
    glBlendFunc( GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA );
    glEnable( GL_BLEND );
    glEnable( GL_DEPTH_TEST );
    glDepthFunc( GL_LEQUAL );
    glLineWidth( 1.0 );
}

void changeSize( int w, int h ) {
    int m = ( w < h ? w : h );
    glViewport( 0, 0, m, m );
    glMatrixMode( GL_PROJECTION );
    glLoadIdentity();
    glOrtho( 0, 0, 50, 50, -1e12, 1e12 );
    glMatrixMode( GL_MODELVIEW );
    glLoadIdentity();
}
```

Listing 19.8 Example of using OpenGL for graphics

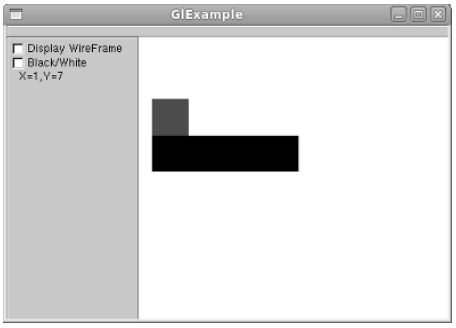


Fig. 19.12 Example of OpenGL, GLUT and GLUI

)

19.2.3 Using OpenGL from Python

In addition to calling OpenGL API functions from compiled C and C++ programs a number of language bindings for OpenGL exist. Python bindings for OpenGL are available using the PyOpenGL bindings. A short example is shown in Listing 19.9.

```
# \file cube.py
# \author Sandeep Koranne, (C) 2010
# \description OpenGL in Python to draw a cube

from OpenGL.GL import *
from OpenGL.GLU import *

import pygame
from pygame.locals import *

WINDOW_SIZE = (400,400)

def resize( w, h ):
    print "Resize called"
    glViewport( 0, 0, w, h )
    glMatrixMode( GL_PROJECTION )
    glLoadIdentity()
    gluPerspective(60.0, float(w)/h, .1, 1.)
    glMatrixMode( GL_MODELVIEW )
    glLoadIdentity()

def OpenGL_Init():
    glEnable( GL_DEPTH_TEST )
    glShadeModel(GL_FLAT)
    glClearColor(1.0, 1.0, 1.0, 0.0)
    glEnable(GL_COLOR_MATERIAL)

    glEnable(GL_LIGHTING)
    glEnable(GL_LIGHT0)
    glLight(GL_LIGHT0, GL_POSITION,  (0, 1, 1, 0))

class DOBJ(object):

    def __init__(self, position, color):

        self.position = position
        self.color = color


    def render(self):
        glColor( self.color )
        glBegin(GL_QUADS)
        glVertex3f( 0.0, 0.0, 1.0 )
        glVertex3f( 1.0, 0.0, 1.0 )
        glVertex3f( 1.0, 1.0, 1.0 )
        glVertex3f( 0.0, 1.0, 1.0 )
        glEnd()

class Model(object):
    def __init__(self):
        self.dobj = DOBJ( (0.0,0.0,0.0), (0.5,0.2,0.3) )

    def render(self):
            self.dobj.render()

def run():
    pygame.init()
    screen = pygame.display.set_mode(WINDOW_SIZE, OPENGL)
```

```
    resize(*WINDOW_SIZE)

    OpenGL_Init()
    model = Model()

    while True:
        glClear( GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT )
        model.render()
        pygame.display.flip()

run()
```

Listing 19.9 OpenGL bindings for Python

19.3 OGRE : OO Graphics Rendering Engine

Ogre is an open-source graphics rendering and scenegraph manager system. Its name is an acronym for Object Oriented Graphics Rendering Engine, and it aims to provide a 3D API which includes a scene graph manager, and can call an user specified 3d rendering library for the actual drawing functions (in our example we have used OpenGL). The key facilities provided by OGRE can be classified as:

1. Scene management: a scene which is to be rendered (per frame, when using continuous rendering) is described to OGRE as placement of geometric shapes, camera locations and viewing angles. Scenes are thus described at a higher level than say point-lists in OpenGL. This also allows application level culling in OGRE. Custom scene manager actions can be taken during rendering by installing a frame listener class,
2. Resource management: OGRE manages the memory for textures, fonts and other resources automatically,
3. Rendering: OGRE implements rudimentary application level culling before calling the low level graphics rendering API to push triangles (or other primitives) onto the screen.

OGRE scene graphs are usually constructed from data stored in files. The data can be directly read in, in a mesh form, or it can be a solid model which is converted to a mesh form during readin. The typical sequence for OGRE scene graph addition is:

```
void ReadData( const char* fileName ) {
  Ogre::Entity *crane = mSceneMgr->createEntity(``Crane'', fileName );
  Ogre::SceneNode* head =
    mSceneMgr->getRootSceneNode()->createChildSceneMode();
  head->attachObject( crane );
}
```

In the above listing, the mesh data for the ‘crane’ object is read from the file. We inquire the rootNode from the scene graph manager, and add the newly created crane object as a child to the root node. The bounding box calculation for the crane and the root node establishes a parent-child relationship, wherein, application level culling

(ALC), will use the bounding box (perhaps using a spatial data structure) to reject the display of scene components which are either not visible to the user (shielded) or have low resolution detail (then faster approximate rendering can be performed for those portions). This facility (of performing automatic ALC and resolution drops) , alongwith scene graph maintenance, are some of the most important features of OGRE, and which require significant code and development time, if done without using an API like OGRE. A related software is Coin3d, which is also used for scene-graph representation and developing 3d applications.

19.4 Graphviz: dot

Graphviz ("graph visualization software") is an open-source software written originally by AT&T Research Labs for drawing graphs. Graph drawing is a major area of research in computer science. The input to the graph drawing is specified in the DOT language script. Consider the following description of a polytope graph.

```
graph POLYTOPE {
0 [shape=box,color=red];
0 -- 1 [color=blue];
0 -- 2 [color=blue];
0 -- 3 [color=blue];
0 -- 4 [color=blue];
1 -- 2 ; 1 -- 3 ; 1 -- 4 ;
2 -- 3 ; 2 -- 4 ; 3 -- 4 ;
}
```

By running the `dot` program on this graph, we can generate the figure as shown in Figure 19.13. The command we used was `dot -Tpng a.dot > a.png`. `dot` supports In addition to drawing pictures of graphs, the algorithms in `dot` have

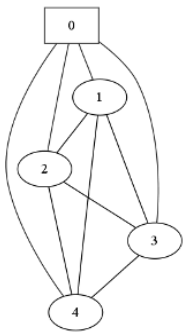


Fig. 19.13 Graph of a polytope generated using dot

also been used in diverse other projects such as Scribus (see Section 2.3) to layout the page. Doxygen uses the `dot` binary to draw pictures of class diagrams.

19.4.1 DOT Language

DOT is the input language for graph drawing software. It is a plain text graph description language. DOT language can be used to specify both directed as well as undirected graphs. An example for DOT to draw an undirected graph of a polytope was shown above. Directed graphs can also be drawn using:

```
digraph Shape {
 Point -> Line  -> Box;
 Line -> Polygon;
}
```

Nodes in the graph can also have attributes, including the node label, node color. Edges can also be annotated with attributes.

19.5 gnuplot

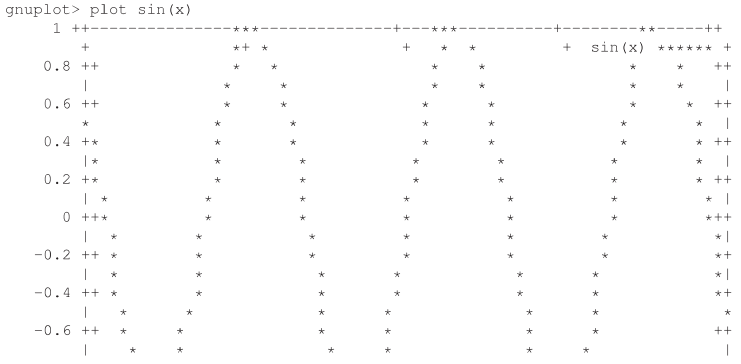
GNUplot is an open-source tool which can generate two and three dimensional plots of data and mathematical functions. Although the name has GNU in it, the software is not related to the GNU project, although it has its own open-source license.

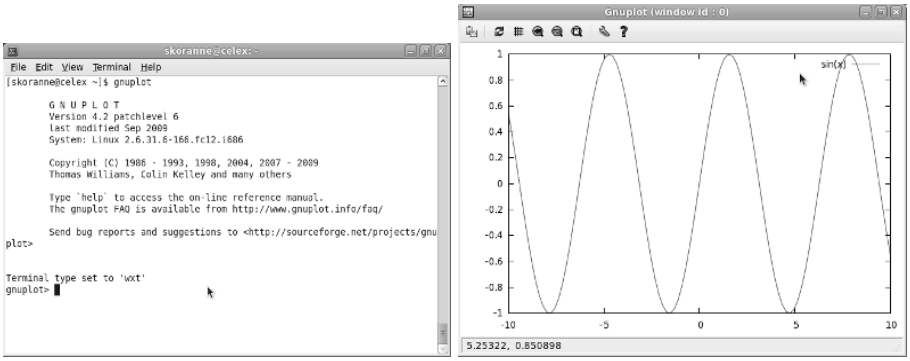
`gnuplot` can produce display directly on X-window terminal, but it can also generate *hardcopy* image files in many formats such as PNG, JPEG, SVG and EPS. Many mathematical software (such as Octave, Maxima, and SAGE) use gnuplot for its plotting needs. Examples of plots rendered using gnuplot are shown in Figure 19.14.

Gnuplot has an extensive help system built into it which can be accessed by typing

```
gnuplot> help
```

The `plot` command of gnuplot is its most powerful, and can be used to plot the contents of data files and mathematical functions. The output format can be changed by using the `set term` command. One such *terminal* is an ASCII terminal as shown below. Gnuplot can also draw figures in xfig (see Section 19.7 and LaTeX(see Section 2.2) formats.





(a) GNUPLOT examples

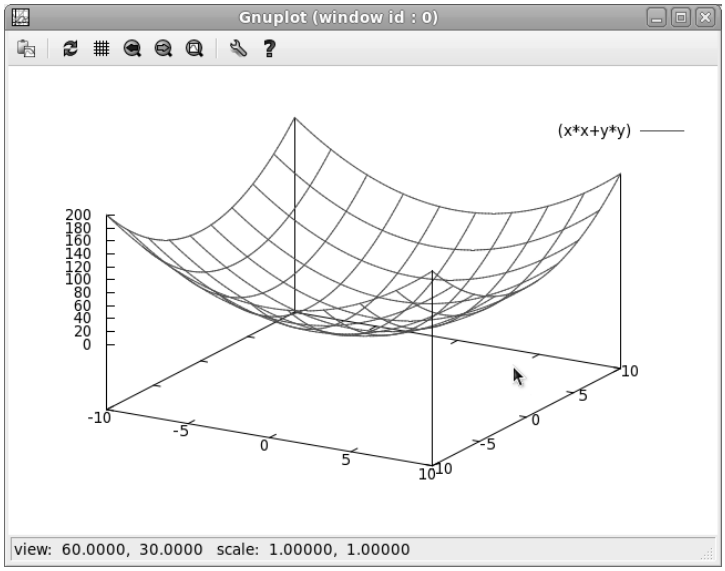


Fig. 19.14 GNUPLOT: scientific plotting software

```
  -0.8 ++   *    *               *   *                *   *              ++
       +    **  *       +         *  *   +              * +*               +
    -1 ++-----**--------+---------***---+--------------***---------------++
      -10               -5               0               5                10

gnuplot>
```

19.6 Grace/Xmgr

Grace is a graphical WYSIWYG 2D plotting tool for the X-Window system and implemented using Motif. Amongst its many features it includes:

1. Many formats for exporting: includes export to EPS, PDF and SVG format,
2. Graphing flexibility: unlimited number of graphs, unlimited number of curves on graphs, color-fill markers and text annotations,

3. Curve fitting: linear and non-linear least-squares fit, calculation of residuals, and region restrictions,
4. Analysis: including FFT, integration and differentiation, histograms, splines and convolution,
5. Data formats: reads netCDF files,
6. Programmability: built-in programmable library, math functions, and user-defined functions via loadable modules.

An example of XMGR/Grace is shown in Figure 19.15.

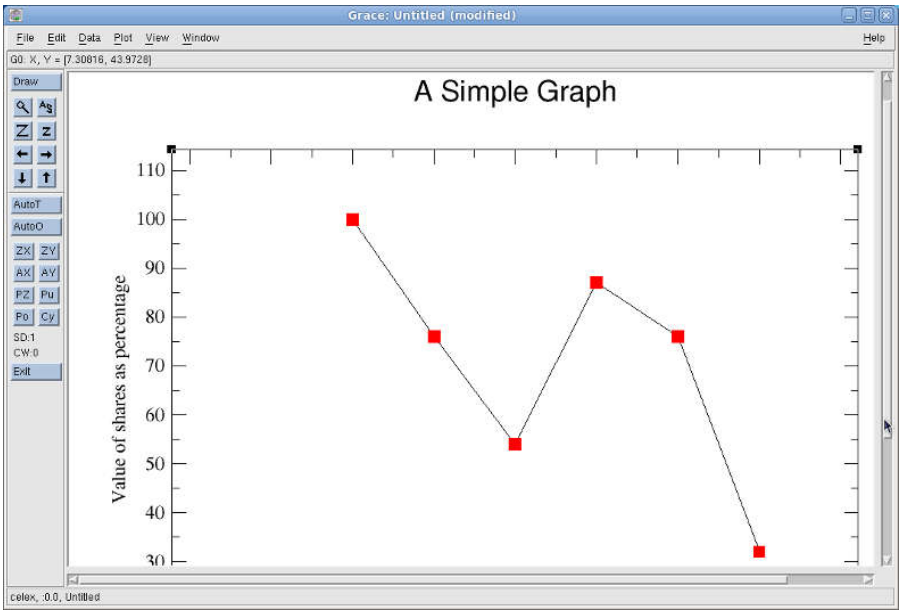
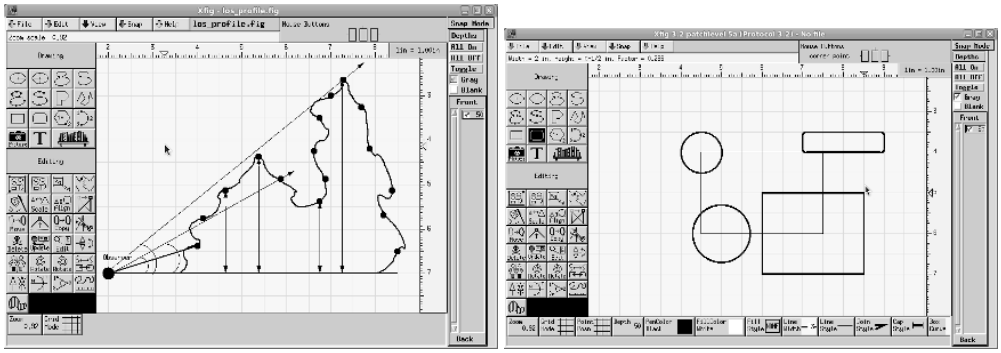


Fig. 19.15 Grace/XMGR plotting software

19.7 Xfig

Xfig is an open-source vector graphics editor running under the X-window system. Xfig is used to draw camera ready pictures. Using xfig it is simple to draw schematics and other technical illustrations containing boxes, lines, circles, ellipses, spline curves and text. Once drawings have been completed an external tool `fig2dev` can be used (or called from within the xfig GUI) to export the image into diverse file formats such as JPEG, EPS and LaTeX.

Like most other UNIX tools, xfig, saves its drawing content as a text-only file (called a '.fig' file). Xfig can also import figures. An example of Xfig generated schematics is shown in Figure 19.16. The text data from Xfig can also be used for non-traditional drawing applications (such as VLSI art work development).

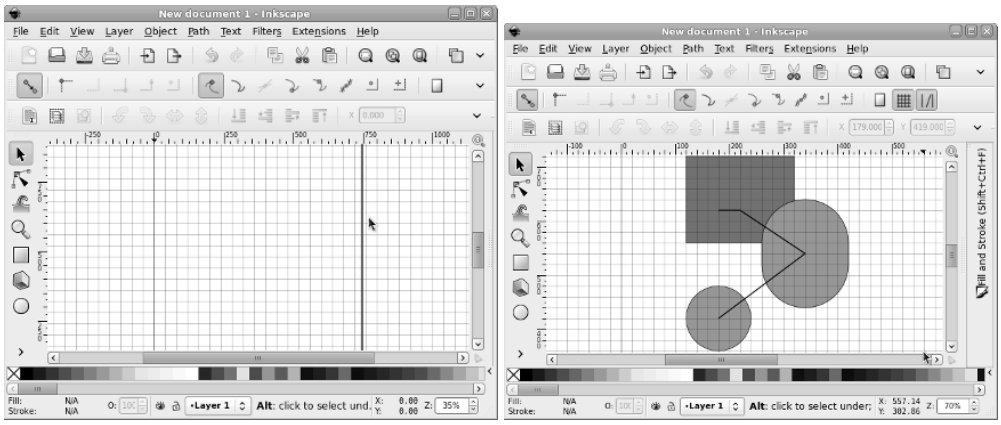


(a)

Fig. 19.16 xfig: vector drawing tool

19.8 Inkscape

Like XFig, Inkscape is also a vector drawing application, but unlike XFig, Inkscape stores its data in XML files, and is fully compliant with XML, SVG and CSS standards. It is designed for artists creating images for the web, and graphic designers, rather than technical illustrators. An example of Inkscape being used to create shapes is shown in Figure 19.17.



(a)

Fig. 19.17 Inkscape: vector drawing tool

Inkscape supports objects such as (i) boxes, (ii) lines, (iii) Bezier curves, (iv) circles and ellipses, and (v) text. Text can be curved around path, and attributes such as font and size can be changed easily. Inkscape also supports rounded rectangles, with customization for fill, stroke, and opaqueness. It also supports the concepts of layers (similar to a raster editor). Unlike other illustration software, Inkscape has

the ability to merge paths together (to perform Boolean AND,OR operations on shapes). Inkscape also has a builtin XML editor to directly edit SVG files document structure, support for LaTeX, and connector tool to create diagrams and flowcharts.

19.9 PovRay : Ray Tracing

POVRAY (Persistence of Vision Ray Tracer) is a *ray tracing* program which can generate photo-realistic renderings of *scenes*. The input to the POVRAY consists of the scene description in its SDL (scene description language). Consider an example as shown below (placed in a file called 'sphere.pov'):

```
/* Example file sphere.pov */
#include "colors.inc"
background { color White }

camera {
  location <0, 2, -3>
  look_at <0, 0, 1>
}

sphere {
  <0, 1, 2>, 1
  texture {
    pigment {
    marble
    turbulence 1
    color_map {
      [0.0 color Gray90]
      [0.8 color Gray60]
      [1.0 color Green]
    }
   }
  }
}
box {
   <-1,0,1.2>, 0.5
  texture {
    pigment {
    marble
    turbulence 1
    color_map {
      [0.0 color Gray90]
      [0.8 color Red]
      [1.0 color Gray20]
    }
  }
 }
}
light_source { <2, 4, -3> color White}
```

This file describes the scene shown in Figure 19.18. The syntax of the POVRAY

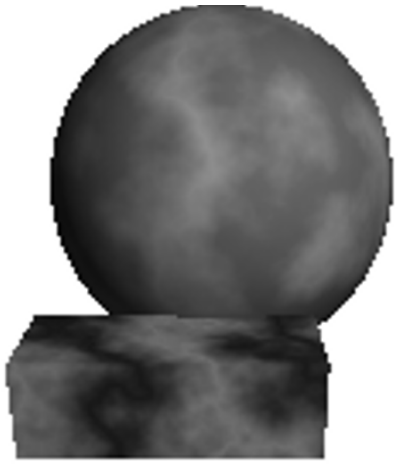


Fig. 19.18 Example of scene rendered using POVRAY

SDL is based on the concept of lights, camera, and objects. Objects can be built up from primitives including, spheres, boxes, torus. Transformations such as translation, scaling and rotation can be applied to objects. Object attributes such as texture, color and size can be specified. The scene also requires the location of a camera and a *look at* direction vector. Moreover, the scene can be illuminated by a light (a light source has a position coordinate, and a color attribute). Figure 19.18 was rendered using POVRAY as:

```
Parsing Options
  Input file: sphere.pov (compatible to version 3.61)
  Remove bounds........On
  Split unions.........Off
  Library paths:
    /usr/local/share/povray-3.6
    /usr/local/share/povray-3.6/ini
    /usr/local/share/povray-3.6/include
Output Options
  Image resolution 320 by 240 (rows 1 to 240, columns 1 to 320).
  Output file: /home/skoranne/POVRAY/sphere.png, 24 bpp PNG
  Graphic display......On  (gamma: 2.2)
  Mosaic preview.......Off
  CPU usage histogram..Off
  Continued trace......Off
Tracing Options
  Quality:  9
  Bounding boxes.......On   Bounding threshold: 3
  Light Buffer.........On
  Vista Buffer.........On   Draw Vista Buffer....Off
  Antialiasing.........Off
  Clock value:    0.000  (Animation off)
```